DATA ENGINEERING REPORT · PROJECT 4

A failure detected within seconds

Real-time supervision of EV chargers:
Kafka, Spark Structured Streaming, Iceberg, Grafana and Terraform

LUMU BAMANA MANASSE

Data engineer

Date	October 2026
Version	1.0.0 (delivered and accepted)
Client	Lumio Recharge (fictional company) · Engagement LUM-2026-002
GitHub repository	https://github.com/MediJr/ev-charging-realtime-streaming
Language	English · *version française disponible dans le dépôt*

Fictional scenario, simulated chargers, real carbon data: the client company and its contacts are invented to simulate a complete professional engagement; charger measurements come from a simulator. Carbon intensity is RTE's real-time éCO2mix data (Open Data Réseaux Énergies, French Open Licence). This project is not commissioned, endorsed or affiliated with RTE.

Document information

Project	Portfolio project 4: streaming, lakehouse and infrastructure as code
Engagement	LUM-2026-002: “Know within seconds that a charger is failing”
Client	Lumio Recharge, operator of 1,200 chargers (fictional), attention of Sophie Bernard and Karim Haddad
Author	Lumu Bamana Manasse, data engineer
Period	October 2026
Data	200 simulated chargers (one measure every 5 s); real RTE carbon intensity
Status	DELIVERED ACCEPTED
Source code	https://github.com/MediJr/ev-charging-realtime-streaming

Version history

Version	Date	Purpose
0.1	2026-10-26	Scoping: brief, meeting minutes, statement of work
0.5	2026-10-26	Streaming platform, lakehouse, supervision, Terraform
0.9	2026-10-27	31-hour endurance run: two defects fixed
1.0.0	2026-10-27	Acceptance (crash, load, catch-up), final report (French and English)

Reading guide

Operations management	1. Executive summary and 10. Results
IT (cloud move)	4. Architecture, 8. GCP infrastructure, 12. Roadmap
Engineer taking over	5. Processing, 6. Lakehouse, 9. Acceptance, 11. Lessons learned

Contents

Document information 2

1 Executive summary 4

2 Context and scoping 5

3 Data 5

4 Architecture 6

5 Spark processing **6**
5.1 Event time, watermark and deduplication 7
5.2 Per-charger state and detecting an absence 7

6 Iceberg lakehouse **7**

7 Supervision **7**

8 Google Cloud infrastructure in Terraform **8**

9 Acceptance **8**
9.1 Reconciliation: every message is explained 9

10 Results **9**

11 Lessons learned **11**

12 Roadmap **12**

1 Executive summary

The need. Lumio Recharge learns that a charger is down when a customer calls, sometimes hours later. Its chargers already send their power every 5 seconds. Operations want an alert **within 30 seconds**, the **live CO_2 of each charging session**, and a supervision screen. The IT department plans to move to Google Cloud next year.

The solution. A **streaming** platform started with a single `docker compose up`. Measures flow through a **Kafka**-compatible event log (Redpanda), are processed in **event time** by **Spark Structured Streaming** (watermark, deduplication, per-charger state), kept in an **Apache Iceberg lakehouse**, enriched with RTE's **real** carbon intensity, and shown in **Grafana**. The target Google Cloud infrastructure is written in **Terraform**.

27 / 27 anomalies detected	**11.6 s** detection delay at the 95th percentile	**0** false alert	**0** message lost or doubled after a hard kill
83,292 messages reconciled one by one	**8.4 s** to catch up 3,654 messages	**≈ 1,440** messages/s (measured capacity)	**22** tests, 5 of them Spark end to end

Key takeaways

- **Q1 · Delay:** all 27 injected anomalies detected in 5.9 s (median) and 11.6 s (95th percentile). Target: 30 s.
- **Q2 · Anomalies:** silent charger, overpower, stuck charger, each opened then resolved, without an alert storm.
- **Q3 · CO_2:** 186 sessions, 4.8 MWh, 211 kg of CO_2 at that evening's real RTE intensity (44 g/kWh).
- **Q4 · Robustness:** Spark killed mid-stream, resumed with no loss and no duplicate. Capacity of about 1,440 messages/s, i.e. some 7,000 chargers: six times the current fleet.

Three defects were found by running the platform for real (31-hour endurance, crash, overload), fixed and covered by tests: an empty timestamp column rejected by Arrow, measure identifiers repeating after a restart, and silent chargers missed during a catch-up.

2 Context and scoping

Client request	Technical consequence
"Detect within seconds"	**Continuous** processing (streaming), not batches
"A charger that stops answering"	Detect an **absence** of events: per-charger state and timeout
4G network delaying or duplicating messages	**Event time**, watermark, deduplication
"Move to Google Cloud without a rewrite"	Open formats (Iceberg), Kafka API, infrastructure in **Terraform**
A crash must lose or double nothing	Checkpoints, idempotent writes

Table 1: From request to technical requirements

Anomaly	Rule (scoping)	Severity
Silent charger	No message for more than 60 s	critical
Overpower	Power above 110% of the rated power	high
Stuck charger	Open session at 0 kW for more than 2 minutes	medium

Table 2: The three anomalies and their rules

3 Data

Charger measures (simulated). 200 chargers on 12 sites, two thirds at 22 kW and one third at 50 kW, one measure every 5 s: power, session energy, status, current session. The simulator **deliberately** injects faults, and publishes each injection to a **ground-truth** topic (`sim.injections`) used to check every detection.

Injected fault	Frequency	What it tests
Silent, overpower, stuck charger	≈ 0.3 per charger per hour	The three anomaly rules
Duplicate message	1%	Deduplication
Delay of 5 to 40 s	1%	Event time
Delay of 3 to 5 min	0.2%	Watermark: dropped and counted

Table 3: Faults injected by the simulator (ADR 005)

Carbon intensity (real). A collector reads RTE's real-time éCO2mix API every 5 minutes, the same source as project 2, and publishes each 15-minute slot to a **compacted** topic: only the latest value of each slot is kept.

4 Architecture

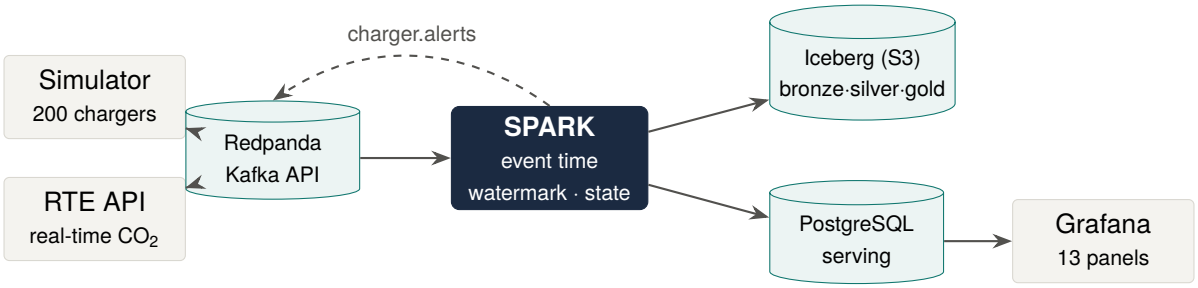


Figure 1: From the charger to the supervision screen

Role	Choice	Why (ADR)
Event log	Redpanda v25.3	Kafka protocol, single binary, light; replaceable by managed Kafka (001)
Processing	Spark 4.0 Structured Streaming	Event time, watermark, fault-tolerant state (001)
Lakehouse	Iceberg 1.10 on S3, JDBC catalog	Transactions, open format read by BigQuery (002)
Serving	PostgreSQL 17	Screen refresh in milliseconds (003)
Supervision	Grafana 12, provisioned as code	Observability standard (003)
Cloud	Terraform, Google provider 8.8	Infrastructure reviewed and versioned before it exists (004)

Table 4: Technical choices

5 Spark processing

A single Spark application runs six queries, each with its own checkpoint:

Query	Trigger	Processing	Destination
bronze	30 s	None: the message as received, with its Kafka position	Iceberg
silver	30 s	JSON parsing, 2-minute watermark, deduplication	Iceberg
monitor	5 s	Per-charger state and anomaly rules	PostgreSQL, Kafka, Iceberg
minutes	30 s	Energy per charger and closed minute, CO_2	PostgreSQL, Iceberg
grid_co2	30 s	RTE carbon intensity	PostgreSQL, Iceberg
injections	30 s	Simulator ground truth	PostgreSQL

Table 5: The six continuous queries

5.1 Event time, watermark and deduplication

A measure is placed according to the time **the charger produced it**, not when it arrived. The 2-minute **watermark** says how long Spark waits for latecomers: beyond it, a measure is dropped, and counted in the micro-batch log. Deduplication uses the measure identifier, `event_id`.

5.2 Per-charger state and detecting an absence

The rules are written in **pure Python** (`lumio_stream.monitor`), tested without Spark. Spark calls them for each charger with its new measures and its stored state (`applyInPandasWithState`). A charger that sends nothing triggers nothing: a 60-second **processing-time timeout**, reset on every measure, raises the "silent charger" alert.

> **Limit found during acceptance: catch-up**
> When Spark catches up a backlog, it crosses several minutes of event time in a few seconds: the processing-time timeout does not fire, and 3 silent chargers were missed. An **event-time safety net** was added: a gap of more than 60 s between two measures, noticed when the charger comes back, opens and closes the alert. It arrives late, but is **never lost**.

6 Iceberg lakehouse

Table	Grain	Content
bronze.telemetry_raw	one Kafka message	Raw message, Kafka partition and offset
silver.telemetry	one measure	Typed, deduplicated fields
gold.charger_minute	charger × minute	Energy, CO_2
gold.alerts	state change	Opened or resolved alert, anomaly and detection times

Table 6: Lakehouse tables (partitioned by day)

Exactly once. Spark's native Iceberg sink is exactly-once by design. For writes made in `foreachBatch`, which only guarantees "at least once", each append is tagged with the micro-batch id in the Iceberg snapshot: a batch replayed after a crash is skipped. Serving writes are UPSERTs.

Maintenance. Streaming writes one small file per micro-batch. A daily script deletes raw measures older than 90 days, compacts small files and expires old snapshots.

7 Supervision

The Grafana screen (13 panels, provisioned as code, anonymous read-only access) first shows **open alerts** sorted by severity, then chargers online, fleet power, RTE intensity, detection delay

at the 95th percentile, throughput, ongoing sessions with their CO_2, and stream health (rows dropped by the watermark). A `StreamingQueryListener` logs every micro-batch to the serving database.

8 Google Cloud infrastructure in Terraform

Resource	Choice
Lakehouse bucket	Paris region, public access prevented, versioning, raw measures deleted after 90 days
BigQuery + BigLake	Dataset for gold tables; BigLake connection to read Iceberg without copying
Service accounts	One per use: processing writes, supervision only reads
Budget alert	50, 90 and 100% of a monthly budget

Table 7: Described infrastructure (ADR 004)

`terraform fmt`, `init -backend=false` and `validate` pass, locally and in continuous integration. A `plan` needs GCP credentials: it will be run with the IT department. Mapping: SeaweedFS becomes Cloud Storage, local Spark becomes Dataproc Serverless (same code), Redpanda becomes managed Kafka (same protocol).

9 Acceptance

Acceptance ran on a **fresh platform**, with Spark hard-killed mid-stream, then a load test.

#	Criterion	Result
C1	Starts with `docker compose up` on a fresh platform	✓
C2	All three injected anomaly types detected (27 / 27, 0 false alert)	✓
C3	Detection delay < 30 s at the 95th percentile (11.6 s)	✓
C4	Duplicates counted once, latecomers counted separately	✓
C5	Spark hard kill: no loss, no duplicate	✓
C6	CO_2 computed with the real RTE intensity	✓
C7	Terraform validated, tests green	✓
C8	No secret in version control	✓

Table 8: Acceptance criteria: 8 out of 8

9.1 Reconciliation: every message is explained

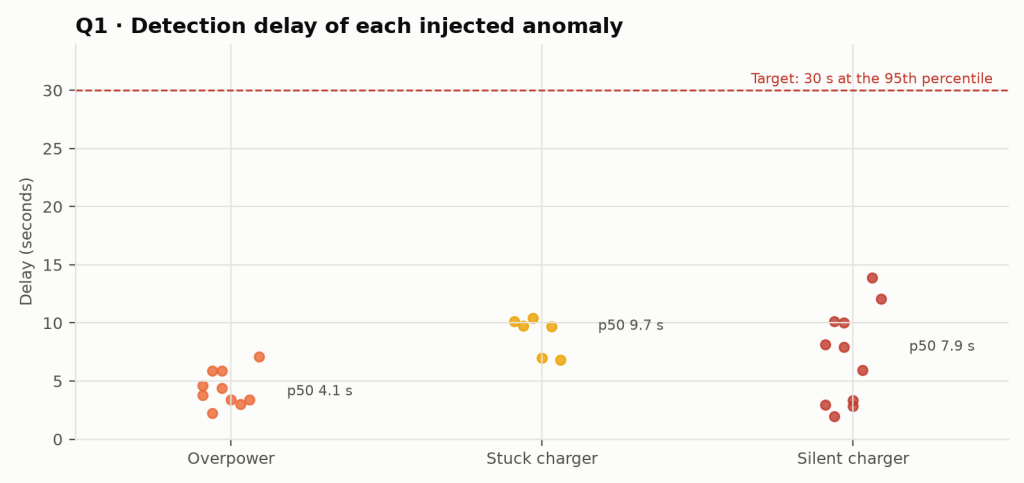
spark/recette.py · messages arrived before 19:46:59

```
bronze messages             :    83,292   (distinct Kafka positions: 83,292)
distinct identifiers        :    82,484   -> duplicates dropped: 808
silver measures             :    82,347   (distinct identifiers: 82,347)
identifiers missing in silver:      137
equation: 83,292 = 82,347 + 808 + 137  -> OK
message read twice by Spark : 0
injections before the cutoff: {'too_late': 164, 'late': 882, 'duplicate': 808}
missing explained by an injected 3-5 min delay: 137 / 137
```

The 808 duplicates dropped are exactly the 808 duplicates injected. The 137 missing measures are all injected delays beyond the watermark. No Kafka message was read twice, despite the hard kill: the first micro-batch after restart is number 41, right after batch 40 committed before the crash, and it caught up 3,654 messages in 8.4 s.

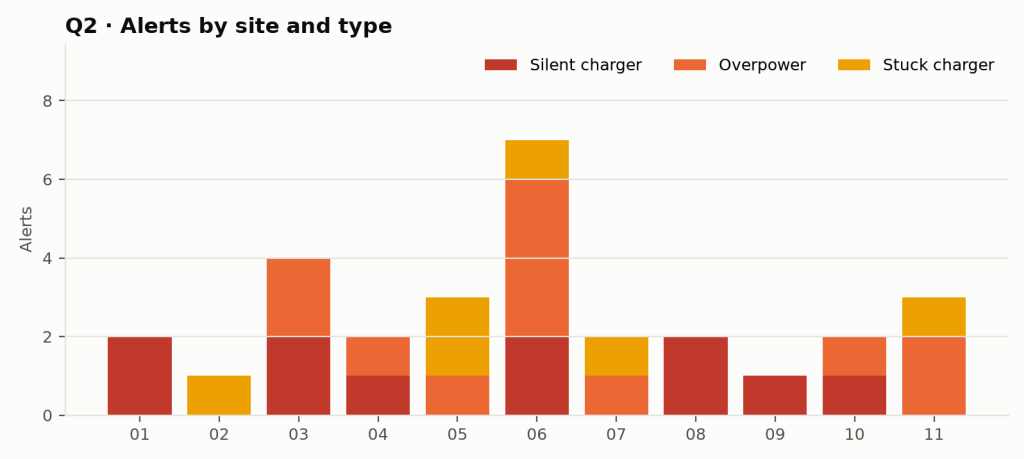
10 Results

Q1 · Detection delay

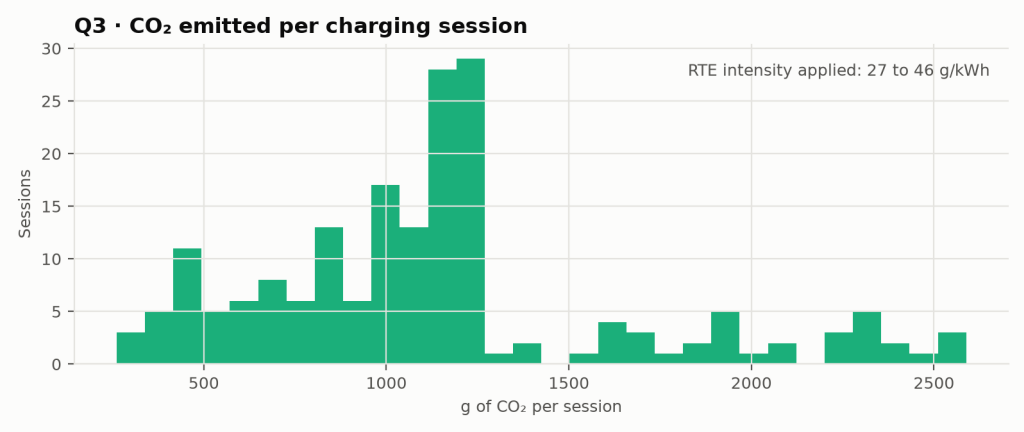


All 27 anomalies injected during the nominal period are detected, with no false alert: overpower in 4.1 s (median), stuck charger in 9.7 s, silent charger in 7.9 s. The 95th percentile is **11.6 s**, the maximum 13.9 s.

Q2 · Alerts by site

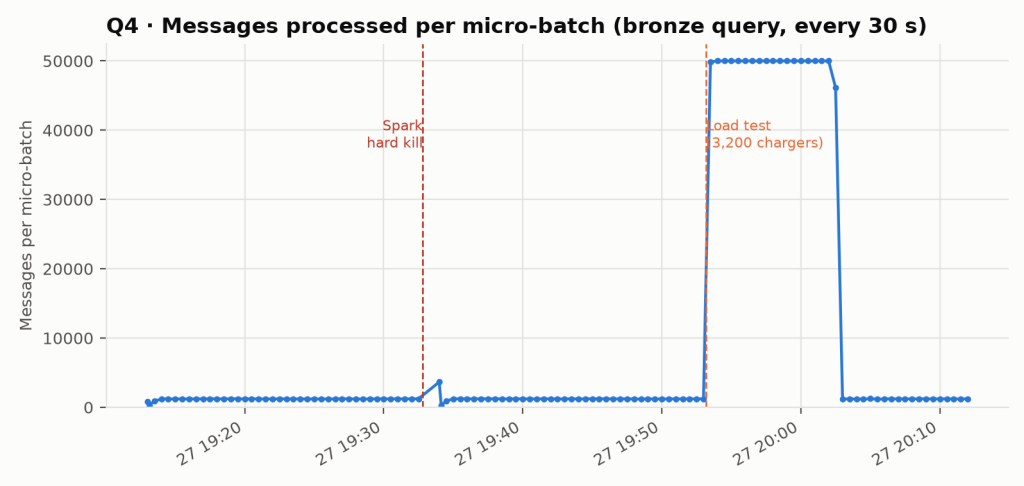


Q3 · CO_2 of charging sessions



186 sessions delivered 4.8 MWh and emitted 211 kg of CO_2, at the grid's real intensity that evening (44 g/kWh on average): a median of 1.1 kg per session. The same computation is shown live for each ongoing session.

Q4 · Crash, catch-up and load



Load test (≈ 2,960 messages/s)	Result
Bronze and silver	Keep up: up to 5,700 and 4,600 messages/s
Stateful monitor	**Bottleneck**: ≈ 1,440 messages/s; detection at 130-160 s during the overload
Stability	No crash; backlog cleared 10 minutes after the overload ended
Capacity	≈ 7,000 chargers at one measure every 5 s, for a fleet of 1,200

Table 9: What the platform handles on a laptop

11 Lessons learned

Defect or incident	Lesson
Spark tests silently skipped	A “skipped” test is not a passed test: check how many tests actually ran
Empty timestamp column rejected by Arrow	The first real batch holds cases no example contained
`event_id`s repeating after a restart	A deduplication key must be unique **over time**, not within one run
First reconciliation wrong	Two tables written by two queries are never at the same instant: compare a fixed set
Silent chargers missed during catch-up	A processing-time timeout makes no sense when time is “fast-forwarded”
Spark stops after laptop sleep	Checkpoints and the restart policy recovered alone; a manual `docker kill` does not

Table 10: What this project taught

12 Roadmap

1. **Capacity**: monitor in Scala (`transformWithState`) or a Dataproc cluster, more Kafka partitions.
2. **Outgoing alerts**: a `charger.alerts` consumer to SMS and ticketing, with deduplication.
3. **GCP deployment**: `terraform plan` with the IT department, then Dataproc Serverless and managed Kafka.
4. **Real chargers**: an OCPP gateway to the `charger.telemetry` topic, with no change to the processing.